BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE

B.Sc. Computer Science · Capstone Project · Academic Year 2025–2026

PROJECT SUMMARY — PRE-VIVA BRIEFING

Solana Local Explorer

A self-hosted, one-command block explorer and real-time transaction indexer for private Solana validators.

SUBMITTED BY

Garvit Dadheech · Shubh Kesharwani

UNDER THE GUIDANCE OF

Dharma Teja

Project Supervisor

UNIQUE VALUE PROPOSITION

Solana Local Explorer is the only self-hosted tool that turns a private Solana validator into a fully browsable, queryable block explorer with a single command — no gRPC knowledge, no database setup, no frontend work, and no need to publish anything to a public cluster.

PROJECT AT A GLANCE

4	1	6	5	38	92
CONTAINERS	COMMAND TO START	DATABASE ENTITIES	REST ENDPOINTS	AUTOMATED TESTS	TOTAL TEST CASES

WHAT IT IS	A complete block explorer and real-time transaction indexer for a Solana chain that no public explorer can see.
WHO IT IS FOR	Solana protocol and dApp developers, QA engineers and infrastructure operators running a private validator.
HOW IT IS USED	`docker compose up --build -d`, then open `http://localhost:3000`.
CORE TECHNIQUE	Consuming the validator's own Geyser event stream over Yellowstone gRPC, rather than polling JSON-RPC.
STACK	solana-test-validator 1.18.26 · Yellowstone Geyser 1.15.3 · TypeScript · Prisma · PostgreSQL 15 · Next.js 16 · Docker Compose
REPOSITORY	`github.com/shubhiscoding/Solana-Local-Indexer-Explorer`
STATUS	Working end to end; 38 of 38 automated tests pass; two defects documented as open (Section 7).

SECTION 1

The Problem

Every Solana developer works against `solana-test-validator`, a complete single-node cluster that runs on their own machine. It starts in seconds, allows unlimited airdrops and can be reset at will. It is also, by construction, invisible to every block explorer in existence — Solscan, SolanaFM and the official Solana Explorer index the public clusters, and no mechanism exists by which a ledger inside one laptop could reach them.

The consequence is that the moment a developer needs to answer an ordinary question — what did my program actually do, which accounts did it touch, how much did it cost, why did it fail — the fast local loop breaks down. The workarounds are all poor: reading validator stdout collapses beyond a handful of transactions; ad-hoc RPC scripts answer one question and must be rewritten for the next; and the most common workaround, deploying to devnet purely to get explorer visibility, converts a fast local loop into a slow remote one and publishes work-in-progress programs.

Solana does expose the right data. The **Geyser plugin interface** streams every transaction out of the validator as it executes, and **Yellowstone gRPC** makes that stream consumable over a standard protocol. But Yellowstone is a transport, not a product: it stores nothing, indexes nothing and answers no questions. Building the rest — schema, indexer, API, interface, deployment — is a multi-day engineering task requiring gRPC, database and frontend competence simultaneously, and it is a task that every team hits independently.

SECTION 2

Unique Value Proposition

UNIQUE VALUE PROPOSITION

Solana Local Explorer is the only self-hosted tool that turns a private Solana validator into a fully browsable, queryable block explorer with a single command — no gRPC knowledge, no database setup, no frontend work, and no need to publish anything to a public cluster.

DIFFERENTIATOR	WHAT IT MEANS IN PRACTICE
One command, zero prerequisites	`docker compose up --build -d` builds and starts the validator, the Geyser plugin, PostgreSQL, the indexer and the web UI in the correct order, gated on health checks. The host needs only Docker — no Rust toolchain, no Solana CLI, no Node runtime, no PostgreSQL installation.
Works where public explorers structurally cannot	The system indexes whatever ledger it is attached to. A local test validator, a private fork, a CI cluster and an air-gapped network are all first-class, because nothing in the pipeline requires the chain to be publicly reachable.
Push-based, not poll-based	Data arrives over the validator's own Geyser stream. There is no polling interval to trade against latency, and the validator is never asked a question it did not volunteer to answer.
Not locked to local	One environment variable (`YELLOWSTONE_ADDR`) re-points the same indexer at any Yellowstone-compatible endpoint, including commercial devnet, testnet and mainnet providers. The local validator is the default, not a limitation.

Positioning against the alternatives

APPROACH	PRIVATE LEDGER	PUSH-BASED	DURABLE & QUERYABLE	SELF-HOSTED	READY-MADE UI	SETUP
Hosted explorers (Solscan, SolanaFM)	✗	n/a	✓	✗	✓	None
RPC polling scripts	✓	✗	Partial	✓	✗	Per query
Raw Yellowstone gRPC	✓	✓	✗	✓	✗	High
Managed platforms (Helius, Dune)	✗	✓	✓	✗	Partial	Low
Solana Local Explorer	✓	✓	✓	✓	✓	**One command**

Every alternative fails at least one criterion that a private-validator workflow makes mandatory. Hosted explorers and managed platforms fail the first, which cannot be engineered around.

SECTION 3

The Solution

The system is four containers orchestrated by a single Docker Compose file, with readiness-gated start-up ordering so that the one-command promise holds reliably on a cold machine.

CONTAINER	ROLE	WHAT IT DOES
`solana-validator`	Blockchain	Runs `solana-test-validator` with the Yellowstone Geyser plugin compiled from source against a matching ABI. Exposes JSON-RPC on 8899 and the gRPC event stream on 10000.
`solana-indexer`	Ingestion	Applies database migrations (with a retry loop), opens the gRPC subscription, decodes each payload, and writes through idempotent upserts so replays cannot duplicate.
`explorer-postgres`	Data	PostgreSQL 15 holding six entities and eleven purpose-built indexes on a persistent named volume.
`solana-explorer`	Presentation	Next.js 16 serving both the REST API and the user interface on port 3000.

How a transaction becomes a page

1. The validator executes a transaction and the Geyser plugin emits it immediately.
2. Yellowstone pushes a `SubscribeUpdate` to the indexer over gRPC — no polling, no RPC round trip.
3. The indexer base-58 decodes the signature and account keys, coerces 64-bit values to `BigInt`, extracts instructions, fees, compute units, program logs and the pre/post balance arrays.
4. It upserts the transaction, one balance row per account key index, and the account registry entries — all keyed on unique constraints, so reprocessing is harmless.
5. The explorer queries the same database through index-backed reads, converts every 64-bit value to a string so JSON cannot lose precision, and renders it.

SECTION 4

Architecture

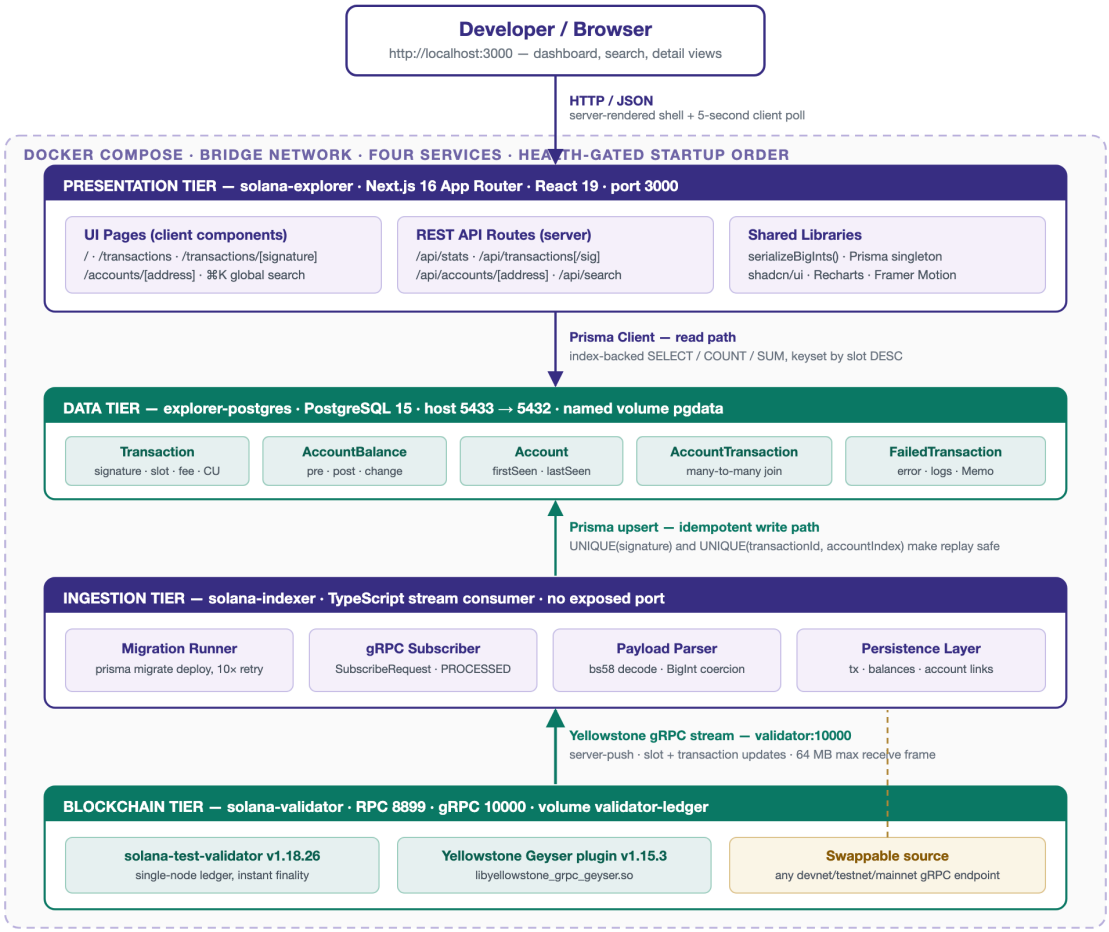


Figure 1 — Four-tier architecture. Data flows strictly upward; the dashed boundary is the Docker Compose network.

The three decisions worth asking about

DECISION	WHY
Geyser stream, not RPC polling	Polling is address-scoped, adds latency proportional to its interval, and loads the node being observed. The Geyser stream delivers every transaction at execution time with none of those costs.
Idempotence by unique constraint, not by application logic	Streams redeliver and indexers restart. Upserting on `signature` and on `(transactionId, accountIndex)` makes duplication impossible by construction rather than by defensive code.
64-bit values as strings at the API boundary	`JSON.stringify` throws on `BigInt`, and coercing to `Number` silently corrupts lamport balances above 2^{53}. A single recursive helper converts at the response boundary and is covered by nine unit tests.

SECTION 5

Key Features

EXPLORER INTERFACE

- **Live dashboard** — total transactions, failures, success rate and latest slot, refreshing every five seconds in place.
- **Activity chart** — stacked success/failure volume over 5 m, 15 m, 1 h or all-time, bucketed client-side so range switching is instant.
- **Transaction list** — paginated, ordered by descending slot, with page state in the URL.
- **Transaction detail** — status, slot, fee, compute units, the ordered account list with links, instructions, memos and, for failures, the error plus full program logs.
- **Account detail** — first/last seen, current balance, and a paginated balance history showing pre-balance, post-balance and signed net change per transaction.
- **Amount filter** — filter balance history by post-balance, pre-balance or net change, as an exact value or a range, in SOL or lamports.
- **⌘K global search** — debounced prefix search across signatures and addresses.
- **Light / dark / system theme**, with reduced-motion support throughout.

PLATFORM

- **Single-command deployment** with health-gated ordering and a clean `down -v` reset.
- **Automatic migrations** on indexer start-up, with a bounded retry loop for a database that is still initialising.
- **Idempotent ingestion** — restarts and stream replays cannot create duplicate rows.
- **Failed transactions preserved** in a dedicated table with the serialised error and the full program log.
- **Five REST endpoints** with a uniform pagination envelope, usable by any script or dashboard.
- **External gRPC support** — one environment variable re-points the indexer at devnet, testnet or mainnet, with optional token authentication.
- **Cross-platform** — macOS (including Apple Silicon), Linux and Windows with WSL2, unmodified.

SECTION 6

Technology Stack

LAYER	TECHNOLOGY	REASON
Blockchain	solana-test-validator 1.18.26	Standard local single-node cluster with instant finality.
Streaming	Yellowstone gRPC Geyser 1.15.3	Production Geyser plugin with a stable Protobuf contract; version-pinned to the validator ABI.
Indexer	Node.js 22 · TypeScript 5.9 · bs58	Shares language and generated client with the explorer; static types catch wire-shape errors.
Persistence	Prisma 5.22 · PostgreSQL 15	64-bit integers, arrays and JSONB, real indexes, migrations checked into the repository.
Web	Next.js 16 · React 19	One deployable serving both the interface and the REST routes.
UI	Tailwind 4 · shadcn/ui · Recharts · Framer Motion	Accessible primitives, declarative charts, reduced-motion-aware transitions.

LAYER	TECHNOLOGY	REASON
Testing	Vitest 4.1.11	Fast TypeScript runner used identically in both packages.
Orchestration	Docker Compose v2 · pnpm 10.27	Health-gated topology; deterministic installs and effective layer caching.

SECTION 7

Testing and Validation

LEVEL	METHOD	CASES	RESULT
Unit — gRPC parsing	AUTOMATED	29	29 / 29 pass
Unit — serialisation	AUTOMATED	9	9 / 9 pass
Integration	MANUAL	12	Documented, reproducible procedures
API	MANUAL	18	Documented, reproducible procedures
Interface	MANUAL	16	Documented, reproducible procedures
Non-functional	MANUAL	8	Documented, reproducible procedures
Total		**92**	**38 automated (all passing) · 54 documented procedures · every requirement traced**

The automated suite deliberately targets the two layers where a silent bug would corrupt data: gRPC payload decoding and `BigInt` serialisation. Both are pure functions, so the suite runs in under a second with no validator, database or network. The remaining 54 cases require a live four-container stack and are therefore written as procedures with exact commands and expected results, reproducible in roughly fifteen minutes.

TWO DEFECTS ARE OPEN AND DISCLOSED

D-06. A transaction consuming exactly zero compute units is recorded as “unknown”, because the extractor uses a JavaScript truthiness check and `0` is falsy. The behaviour is deliberately pinned by a unit test with an explanatory comment so it cannot regress silently; the fix is a one-line change deferred to avoid a late edit to the ingestion hot path.

D-09. An address last seen before balance tracking was introduced has no balance rows, so its account page reports “not found”. This is a consequence of forward-only indexing and is documented in the user-facing material; it resolves as soon as the address appears in a new transaction.

SECTION 8

Results, Limitations and Future Work

All six project objectives are met: a containerised validator with a compiled Geyser plugin; a resilient, idempotent indexer; a normalised, migrated and indexed schema; a documented REST API and a complete interface; single-command orchestration with a clean reset; and a validated test suite with a full traceability matrix.

KNOWN LIMITATIONS

- **No historical backfill** — indexing starts at connection time.

FUTURE WORK, IN PRIORITY ORDER

- Fix defect D-06 and extract the parsing handler so the tests exercise the production code path directly.

- **Instructions are not decoded** — raw program index and base-58 data, not named methods; decoding needs per-program IDLs.
- **Lamports only** — SPL token balances are not interpreted.
- **Single indexer instance**; no horizontal scaling.
- **No authentication** — intended for localhost or a trusted network.
- **Sequential account writes** — the throughput limit if pointed at a busy public stream.

- Replace the five-second dashboard poll with server-sent events.
- IDL registry for instruction decoding — additive, no re-indexing needed.
- SPL token balance tracking from metadata already present in the payload.
- Opt-in historical backfill worker writing through the same idempotent upserts.
- Batched account writes; program-level analytics; CI running both suites plus a Compose smoke test.

SECTION 9

Viva Demonstration Guide

A ten-minute path through the system, in the order the authors will demonstrate it.

STEP	ACTION	WHAT IT DEMONSTRATES
1	`docker compose up --build -d`	The single-command claim. Four services build and start with health-gated ordering.
2	`docker compose ps`	All four running; validator and PostgreSQL reporting `(healthy)`.
3	`docker compose logs -f indexer`	Migrations applied, gRPC subscription established, and live `Indexed transaction …` lines as the validator produces blocks.
4	Open `http://localhost:3000`	The dashboard, updating live without a page reload; the activity chart across ranges.
5	Airdrop and transfer inside the validator container	The new transaction appears within one five-second refresh — the end-to-end latency claim.
6	Open the new transaction's detail page	Accounts, instructions, fee and compute units, with each account linked to its own page.
7	Follow an account link, then apply an amount filter	Balance history with signed net change; filtering by field, unit and range, with the filter reflected in the URL.
8	Press ⌘K and search a signature prefix	Debounced prefix search across signatures and addresses.
9	`curl localhost:3000/api/stats | jq`	The programmatic surface, and that 64-bit values are strings rather than lossy numbers.
10	`docker restart solana-indexer`, then re-check row counts	Idempotence: the indexer resumes and no duplicate signatures appear.
11	`pnpm test` in both packages	38 automated tests passing in under a second.

Questions the authors expect, and where the answers are

QUESTION	REFERENCE IN THE FINAL PROJECT REPORT
Why a Geyser stream instead of polling RPC?	Section 2.2 and decision D-1 in Table 4.1
What stops a stream replay from duplicating rows?	Decision D-4; schema constraints in Section 4.3; test NF-04
Why are slots and fees strings in the JSON?	Decision D-8; Section 5.6.1; tests UT-SER-01 – 09
Why is `accountAddress` not a foreign key?	Decision D-7 — a deliberate trade of strict referential integrity for single-pass writes
How do you know the parsing is correct?	Section 6.3.2 — 29 unit tests, including malformed and missing input
What does not work, and what would you do next?	Sections 6.8, 8.2 and 9.2

ACCOMPANYING SUBMISSION

Final Project Report (61 pages, with full design rationale, the 92-case test catalogue and four appendices) · source archive · GitHub link · user manual · installation guide · standalone test and validation report · presentation deck.